

Explain Integer Break Logic

The **real intuition** of this code is:

For every possible first cut i , we have two choices for the remaining part $n-i$:

1. **Break it further** $\rightarrow i * \text{solve}(n-i)$
2. **Don't break it anymore** $\rightarrow i * (n-i)$

Then take the maximum.

This is the core idea behind **Integer Break + DP**.

1. What is the actual problem?

For example:

```
text
n = 10
```

We need to break 10 into **at least two positive integers** such that their product is maximum.

Some possibilities:

```
text
10 = 5 + 5      → 5 × 5 = 25
10 = 3 + 7      → 3 × 7 = 21
10 = 2 + 8      → 2 × 8 = 16
10 = 3 + 3 + 4  → 3 × 3 × 4 = 36
10 = 2 + 2 + 3 + 3 → 36
```

Answer:

```
text
36
```

2. The most important intuition

Suppose you're solving:

```
text
solve(10)
```

You decide to make the **first cut** at $i = 3$.

So:

```
text
10
↓
3 + 7
```

Now you have:

```
text
3 × 7
```

But here's the important question:

What should we do with 7?

We have **two choices**.

Choice 1 – Don't break 7

```
text
3 × 7 = 21
```

This is:

```
C++
int not_break_n = i * (n - i);
```

Choice 2 – Break 7 further

Maybe:

text

$$7 = 3 + 4$$

Then:

text

$$3 \times 3 \times 4 = 36$$

This is:

C++

```
int break_n = i * solve(n - i, dp);
```

So the code is essentially asking:

"After making my first cut, should I leave the remaining piece alone, or should I recursively break it?"

That's the **real intuition**.

3. Why do we need `break` and `not_break` separately?

This is the part you should understand deeply.

Consider:

text

$$\begin{aligned} n &= 10 \\ i &= 3 \end{aligned}$$

We split:

text

$$10 \rightarrow 3 + 7$$

Now:

text

$$3 \times 7$$

Option A: Don't break 7

text

$$\begin{aligned} 3 \times 7 \\ = 21 \end{aligned}$$

Code:

C++

```
not_break_n = i * (n-i);
```

Option B: Break 7

Suppose the best way to break 7 is:

text

$$7 \rightarrow 3 + 4$$

Then:

text

$$\begin{aligned} 3 \times (3 \times 4) \\ = 36 \end{aligned}$$

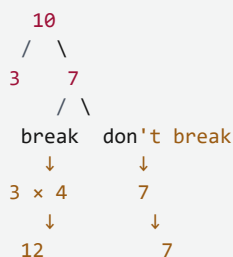
Code:

C++

```
break_n = i * solve(n-i, dp);
```

So:

text



Then multiply by 3:

break: $3 \times 12 = 36$
not break: $3 \times 7 = 21$

Therefore:

text

$\max(36, 21) = 36$

4. Why can't we just use `solve(n-i)`?

You might think:

C++

```
int break_n = i * solve(n-i, dp);
```

Why not simply do this?

Because `solve(x)` assumes that `x` must be broken.

But after choosing the first piece `i`, the remaining `n-i` doesn't necessarily need to be broken.

For example:

text

$10 \rightarrow 5 + 5$

If we choose:

C++

```
i = 5
n-i = 5
```

The best result is:

text

$5 \times 5 = 25$

We need the option:

text

`not_break = 5 x 5 = 25`

If we only did:

C++

```
5 * solve(5)
```

then:

text

`solve(5) = 6`

because:

text

$5 \rightarrow 2 + 3$
 $2 \times 3 = 6$

Then we'd get:

text

$5 \times 6 = 30$

But 30 is invalid for the original integer-break interpretation because that would correspond to:

text

$10 = 5 + 2 + 3$

Actually, that decomposition is valid and gives 30 — but the key issue is that `solve(5)` as implemented returns 6, which means "best product after breaking 5", while 5 itself may be the optimal **unbroken factor** in a particular decomposition.

So we need both possibilities.

5. Think of `solve(n)` as a decision tree

The function:

C++

```
solve(n)
```

means:

"What is the maximum product I can obtain from n if I am allowed to break it?"

For every possible first piece:

C++

```
for(int i = 1; i < n; i++)
```

we make:

text

n
↓
 $i + (n-i)$

Then the remaining part has two possibilities:

text

```
      n
      |
    choose i
      |
    i + (n-i)
    /      \
  don't break  break
    |          |
  i*(n-i)    i*solve(n-i)
```

Then:

C++

```
maxprod = max(maxprod, max(break_n, not_break_n));
```

6. Let's actually trace `solve(5)`

Start:

text

```
solve(5)
```

Try every possible i .

$i = 1$

text

$5 \rightarrow 1 + 4$

Don't break:

text

$1 \times 4 = 4$

Break 4:

text

$1 \times \text{solve}(4)$

`solve(4)` = 4

So:

text

$$1 \times 4 = 4$$

Best:

text

$$4$$

i = 2

text

$$5 \rightarrow 2 + 3$$

Don't break:

text

$$2 \times 3 = 6$$

Break 3:

text

$$2 \times \text{solve}(3)$$

$$\text{solve}(3) = 2$$

So:

text

$$2 \times 2 = 4$$

Best:

text

$$6$$

i = 3

text

$$5 \rightarrow 3 + 2$$

Don't break:

text

$$3 \times 2 = 6$$

Break 2:

text

$$3 \times \text{solve}(2)$$

$$\text{solve}(2) = 1$$

So:

text

$$3 \times 1 = 3$$

Best:

text

$$6$$

Therefore:

text

$$\text{solve}(5) = 6$$

7. Now look at `solve(10)`

One branch:

text

$$10 \rightarrow 3 + 7$$

We have:

```
C++
not_break_n = 3 * 7 = 21
```

And:

```
C++
break_n = 3 * solve(7)
```

We know:

```
text
solve(7) = 12
```

because:

```
text
7 → 3 + 4
3 × 4 = 12
```

Therefore:

```
text
break_n = 3 × 12
        = 36
```

So this branch gives:

```
text
max(21, 36)
= 36
```

8. What exactly does `solve(n)` return?

This is extremely important.

Your function:

```
C++
int solve(int n, vector<int>& dp)
```

does **not** return:

"The best product if I don't break n ."

It returns:

The maximum product obtainable by breaking n into at least two positive integers.

For example:

```
text
solve(2) = 1

2 → 1 + 1
1 × 1 = 1
```

```
text
solve(3) = 2

3 → 1 + 2
1 × 2 = 2
```

```
text
solve(4) = 4

4 → 2 + 2
2 × 2 = 4
```

```
text
solve(5) = 6

5 → 2 + 3
2 × 3 = 6
```

9. Why is the base case `n == 1`?

```
C++
if(n == 1) return 1;
```

This is mainly because when you recursively break something, eventually you can reach `1`.

For example:

```
text
solve(4)
  ↓
solve(3)
  ↓
solve(2)
  ↓
solve(1)
```

Returning `1` gives a multiplicative identity:

```
text
i × 1 = i
```

It allows the recursion to terminate cleanly.

10. What is DP actually doing here?

Without DP, the same values get calculated again and again.

For example:

```
text
solve(10)
├── solve(9)
│   ├── solve(8)
│   │   └── solve(...)
│   └── solve(8)
│       └── ...
└── solve(7)
    └── ...
```

`solve(8)`, `solve(7)`, etc. can be requested multiple times.

So:

```
C++
if(dp[n] != -1)
    return dp[n];
```

means:

"Have I already solved this value of `n`?"

If yes, don't calculate it again.

Then:

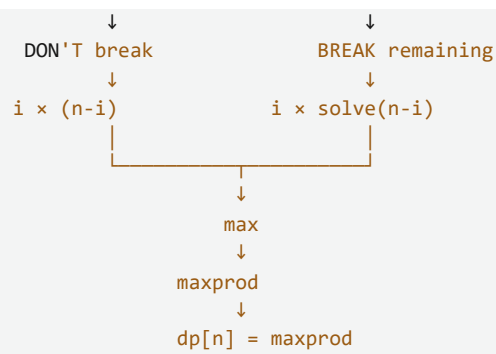
```
C++
return dp[n] = maxprod;
```

stores the answer.

11. The entire logic in one picture

Think about your algorithm like this:

```
text
          solve(n)
            |
    Try every first cut i
            |
          n = i + (n-i)
            |
    ┌───────────┴───────────┐
```



That's the entire algorithm.

12. The subtle point you should remember

The names can actually be clearer.

Instead of:

```
C++
break_n
not_break_n
```

I would mentally read them as:

```
C++
int splitFurther = i * solve(n-i, dp);
int keepAsWhole = i * (n-i);
```

Because the decision is really:

```
text
Remaining piece (n-i)
  |
  |— Keep it whole
  |— Split it further
```

That makes the code much easier to understand.

In one sentence:

At every cut, we decide whether the remaining portion should remain one factor or should itself be optimized recursively, and DP remembers the best answer for every remaining size.

That's the key intuition behind this solution.